C++ Parametric Number Type Aliases

ISO/IEC JTC1 SC22 WG21 P0102R0 - 2015-09-27

Lawrence Crowl, Lawrence@Crowl.org

Introduction

With the introduction of template non-type parameters, template class partial specialization and constexpr functions, it becomes possible to compute the size needed of a built-in numeric type at compile time. However, to make that computation effective, we need a mechanism to map the computed size to a built-in type.

We may need to eventually support three arithmetic type categories (signed integer, unsigned integer, and floating point) over two bases (binary and decimal). The implication is that any naming should include both category and base.

Solution

We propose a set of template type aliases that take the number of bits required of a type as a template parameter and provide a built-in type.

An alternate approach is to specify the number of digits. These approaches are nearly equivalent in binary, but will be substantially different in decimal, and even more so in floating-point where the exponent digits might also be specified.

We sometimes need types that hold "at least as many bits" and "exactly this many bits". So, we provide two facilities, one for each. In addition, we occasionally need to specify the fastest type for a given number of bits.

```
exact_2uint<32> a; // a built-in binary unsigned integer of exactly 32 bits
least_2int<19> a; // a built-in binary signed integer of at least 19 bits
fast_2int<7> a; // the fastest built-in binary signed integer of at least 7 bits
```

Putting the base before the type name seems a bit strange, but it avoids confusion with the current practice of putting the size after the type name.

It is not unusual for two built-in types to have the same size. We propose a "round towards int" rule when selecting the type. This approach reduces literal promotion artifacts.

Most, if not all, C++ implementations of binary signed integer types use a two's complement representation. We propose to require a two's complement representation for the types referenced by the alias. In this approach, we follow the lead of `<cstdint>`, but extend it to require the "least" types to also be two's complement.

Furthermore, most C++ implementation's floating point types use an IEEE (IEEE 754-2008 aka ISO/IEC/IEEE 60559:2011) representation. (They may or may not implement full operational semantics.) To ensure machine-to-machine portability, we propose to require an IEEE representation. We follow the lead of N3626 Floating-Point Typedefs Having Specified Widths in the exact specification and size.

```
exact_2ieeefloat<64> a; // a built-in float of exactly 64 bits
fast_2ieeefloat<32> a; // the fastest built-in float of at least 32 bits
least_2ieeefloat<80> a; // a built-in float of at least 80 bits
```

Because the built-in types available on a platform may not meet the requirements specified here, we make these aliases conditionally supported.

Finally, we need a mechanism to determine the largest of each type supported, in bits. Ideally, the mechanism could affect conditional compilation. Therefore, we add macros. If a type alias is not supported, the value of the macro should be zero.

```
#if MAX_BITS_2INT >= 64 // bits in the largest signed integer
#if MAX_BITS_2UINT >= 64 // bits in the largest unsigned integer
#if MAX_BITS_2IEEEFLOAT >= 80 // bits in the largest float
```

Open Issues

Should we provide a mechanism to indicate the degree of hardware implementation? Possible degrees are full hardware implementation, full microcode implementation, hardware or microcode implementation with trapping for unusual values, or full software implementation. Since binaries area often shipped among different implementations of the binary, a run-time query would be necessary to be effective.

What headers should contain these type aliases? The integer types are perhaps appropriate to `<cstdint>`. We anticipate a floating-point equivalent to `<cstdint>`, but it does not yet exist. N3626 suggests using `<cfloat>`, but that header seems more specific to describing existing floating types, as `<climits>` describes existing integer types. Since it is easier to merge headers than split them, we choose to specify a `<cstdfloat>` header.

This paper provides no mechanism for specifying literals of the corresponding type. Since types are computed, literals are unlikely to appear in the source.

Wording

The following wording changes are relative to N4527.

17.6.1.3 Frestanding implementations [compliance]

Add the following entry to table 16.

18.4+	Floating-point types	`<cstdfloat>`

18.1 General [support.general]

Add the following entry to table 29.

18.4+	Floating-point types	`<cstdfloat>`

18.4.1 Header `<cstdint>` [cstdint.syn]

Add the following entries to the synopsis before paragraph 1.

```
namespace std {
  template<int bits> alias exact_2int = implementation-defined;
  template<int bits> alias fast_2int = implementation-defined;
  template<int bits> alias least_2int = implementation-defined;
  template<int bits> alias exact_2uint = implementation-defined;
  template<int bits> alias fast_2uint = implementation-defined;
  template<int bits> alias least_2uint = implementation-defined;
}
#define MAX_BITS_2INT implementation-defined;
#define MAX_BITS_2UINT implementation-defined;
```

18.4.2 Parametric types [cstdint.parametric]

Add a new section with the following paragraphs.

The aliases below are conditionally supported. The macro `MAX_BITS_2INT` shall give the largest integer size (in bits)

supported by the aliases. [*Note:* All variants support the same sizes. —*end note*] If these aliases are not supported, the value shall be `0`. Any parameter to the alias shall be in the range 1 to `MAX_BITS_2INT`.

`template<int bits> alias exact_2int`

> The alias `exact_2int` refers to a built-in signed binary integer type of exactly `bits` bits. If there are two types of the same size, it refers to the type that is closest to `int` in promotion order. The type must represent negative values with two's-complement representation.

`template<int bits> alias fast_2int`

> The alias `fast_2int` refers to the fastest built-in signed binary integer type of at least `bits` bits. If there are two types of the same size, it refers to the type that is closest to `int` in promotion order. The type must represent negative values with two's-complement representation.

`template<int bits> alias least_2int`

> The alias `least_2int` refers to the smallest built-in signed binary integer type of at least `bits` bits. If there are two types of the same size, it refers to the type that is closest to `int` in promotion order. The type must represent negative values with two's-complement representation.

`template<int bits> alias exact_2uint`

> The alias `exact_2uint` refers to a built-in unsigned binary integer type of exactly `bits` bits. If there are two types of the same size, it refers to the type that is closest to `unsigned int` in promotion order.

`template<int bits> alias fast_2uint`

> The alias `fast_2uint` refers to the fastest built-in unsigned binary integer type of at least `bits` bits. If there are two types of the same size, it refers to the type that is closest to `unsigned int` in promotion order.

`template<int bits> alias least_2uint`

> The alias `least_2uint` refers to the smallest built-in unsigned binary integer type of at least `bits` bits. If there are two types of the same size, it refers to the type that is closest to `unsigned int` in promotion order.

18.4+ Floating-point types [cstdfloat]

After section 18.4, add a new section. It has no direct contents.

18.4+.1 Header `<cstdfloat>` [cstdfloat.syn]

Add a new section.

```
namespace std {
  template<int bits> alias exact_2ieeefloat = implementation-defined;
  template<int bits> alias fast_2ieeefloat = implementation-defined;
  template<int bits> alias least_2ieeefloat = implementation-defined;
  template<int bits> alias exact_10ieeefloat = implementation-defined;
  template<int bits> alias fast_10ieeefloat = implementation-defined;
  template<int bits> alias least_10ieeefloat = implementation-defined;
}
#define MAX_BITS_2IEEEFLOAT implementation-defined;
#define MAX_BITS_10IEEEFLOAT implementation-defined;
```

18.4+.2 Parametric types [cstdfloat.parametric]

Add a new section with the following paragraphs.

The aliases below are conditionally supported. The macro `MAX_BITS_2IEEEFLOAT` shall give the largest binary floating-point size (in bits) supported by the aliases. [*Note:* All variants support the same sizes. —*end note*] If none of these aliases are supported, the value shall be `0`. The parameter to the alias shall be in the range 1 to `MAX_BITS_2IEEEFLOAT`.

`template<int bits> alias exact_2ieeefloat`

The alias `exact_2ieeefloat` refers to a built-in binary floating-point type of exactly `bits` bits. If there are two types of the same size, it refers to the type that is closest to `double` in promotion order. The type must use IEEE representation.

`template<int bits> alias fast_2ieeefloat`

The alias `fast_2ieeefloat` refers to the fastest built-in binary floating-point type of at least `bits` bits. If there are two types of the same size, it refers to the type that is closest to `double` in promotion order. The type must use IEEE representation.

`template<int bits> alias least_2ieeefloat`

The alias `least_2ieeefloat` refers to the smallest built-in binary floating-point type of at least `bits` bits. If there are two types of the same size, it refers to the type that is closest to `double` in promotion order. The type must use IEEE representation.

The aliases below are conditionally supported. The macro `MAX_BITS_10IEEEFLOAT` shall give the largest decimal floating point size (in bits) supported by the aliases. [*Note:* All variants support the same sizes. —*end note*] If none of these aliases are supported, the value shall be `0`. The parameter to the alias shall be in the range 1 to `MAX_BITS_10IEEEFLOAT`.

`template<int bits> alias exact_10ieeefloat`

The alias `exact_10ieeefloat` refers to a built-in decimal floating-point type of exactly `bits` bits. If there are two types of the same size, it refers to the type that is closest to `double` in promotion order. The type must use IEEE representation.

`template<int bits> alias fast_10ieeefloat`

The alias `fast_10ieeefloat` refers to the fastest built-in decimal floating-point type of at least `bits` bits. If there are two types of the same size, it refers to the type that is closest to `double` in promotion order. The type must use IEEE representation.

`template<int bits> alias least_10ieeefloat`

The alias `least_10ieeefloat` refers to the smallest built-in decimal floating-point type of at least `bits` bits. If there are two types of the same size, it refers to the type that is closest to `double` in promotion order. The type must use IEEE representation.

Implementation

The following implementation will work on some current systems.

```
#define MAX_BITS_2INT 64

template<int bits> struct exact_2int_special;
template<> struct exact_2int_special<8>  { typedef std::int8_t  type; };
template<> struct exact_2int_special<16> { typedef std::int16_t type; };
template<> struct exact_2int_special<32> { typedef std::int32_t type; };
template<> struct exact_2int_special<64> { typedef std::int64_t type; };
template<int bits> using exact_2int = exact_2int_special<bits>::type;

template<int bits> struct fast_2int_special;
template<> struct fast_2int_special<1> { typedef std::int8_t  type; };
template<> struct fast_2int_special<2> { typedef std::int16_t type; };
template<> struct fast_2int_special<3> { typedef std::int32_t type; };
template<> struct fast_2int_special<4> { typedef std::int32_t type; };
template<> struct fast_2int_special<5> { typedef std::int64_t type; };
template<> struct fast_2int_special<6> { typedef std::int64_t type; };
template<> struct fast_2int_special<7> { typedef std::int64_t type; };
template<> struct fast_2int_special<8> { typedef std::int64_t type; };
template<int bits> using fast_2int = fast_2int_special<(bits+7)/8>::type;

template<int bits> struct least_2int_special;
template<> struct least_2int_special<1> { typedef std::int8_t  type; };
template<> struct least_2int_special<2> { typedef std::int16_t type; };
template<> struct least_2int_special<3> { typedef std::int32_t type; };
template<> struct least_2int_special<4> { typedef std::int32_t type; };
```

```
template<> struct least_2int_special<5> { typedef std::int64_t type; };
template<> struct least_2int_special<6> { typedef std::int64_t type; };
template<> struct least_2int_special<7> { typedef std::int64_t type; };
template<> struct least_2int_special<8> { typedef std::int64_t type; };
template<int bits> using least_2int = least_2int_special<(bits+7)/8>::type;

template<int bits> struct exact_2uint_special;
template<> struct exact_2uint_special<8>  { typedef std::uint8_t  type; };
template<> struct exact_2uint_special<16> { typedef std::uint16_t type; };
template<> struct exact_2uint_special<32> { typedef std::uint32_t type; };
template<> struct exact_2uint_special<64> { typedef std::uint64_t type; };
template<int bits> using exact_2uint = least_2uint_special<bits>::type;

template<int bits> struct fast_2uint_special;
template<> struct fast_2uint_special<1> { typedef std::uint8_t  type; };
template<> struct fast_2uint_special<2> { typedef std::uint16_t type; };
template<> struct fast_2uint_special<3> { typedef std::uint32_t type; };
template<> struct fast_2uint_special<4> { typedef std::uint32_t type; };
template<> struct fast_2uint_special<5> { typedef std::uint64_t type; };
template<> struct fast_2uint_special<6> { typedef std::uint64_t type; };
template<> struct fast_2uint_special<7> { typedef std::uint64_t type; };
template<> struct fast_2uint_special<8> { typedef std::uint64_t type; };
template<int bits> using fast_2uint = fast_2uint_special<(bits+7)/8>::type;

template<int bits> struct least_2uint_special;
template<> struct least_2uint_special<1> { typedef std::uint8_t  type; };
template<> struct least_2uint_special<2> { typedef std::uint16_t type; };
template<> struct least_2uint_special<3> { typedef std::uint32_t type; };
template<> struct least_2uint_special<4> { typedef std::uint32_t type; };
template<> struct least_2uint_special<5> { typedef std::uint64_t type; };
template<> struct least_2uint_special<6> { typedef std::uint64_t type; };
template<> struct least_2uint_special<7> { typedef std::uint64_t type; };
template<> struct least_2uint_special<8> { typedef std::uint64_t type; };
template<int bits> using least_2uint = least_2uint_special<(bits+7)/8>::type;

#define MAX_BITS_2IEEEFLOAT 80

template<int bits> struct exact_2ieeefloat_special;
template<> struct exact_2ieeefloat_special<32> { typedef float type; };
template<> struct exact_2ieeefloat_special<64> { typedef double type; };
template<> struct exact_2ieeefloat_special<80> { typedef long double type; };
template<int bits> using exact_2ieeefloat
  = least_2ieeefloat_special<bits>::type;

template<int bits> struct fast_2ieeefloat_special;
template<> struct fast_2ieeefloat_special<1> { typedef float type; };
template<> struct fast_2ieeefloat_special<2> { typedef float type; };
template<> struct fast_2ieeefloat_special<3> { typedef double type; };
template<> struct fast_2ieeefloat_special<4> { typedef double type; };
template<> struct fast_2ieeefloat_special<5> { typedef long double type; };
template<int bits> using fast_2ieeefloat
  = fast_2ieeefloat_special<(bits+15)/16>::type;

template<int bits> struct least_2ieeefloat_special;
template<> struct least_2ieeefloat_special<1> { typedef float type; };
template<> struct least_2ieeefloat_special<2> { typedef float type; };
template<> struct least_2ieeefloat_special<3> { typedef double type; };
template<> struct least_2ieeefloat_special<4> { typedef double type; };
template<> struct least_2ieeefloat_special<5> { typedef long double type; };
template<int bits> using least_2ieeefloat
  = least_2ieeefloat_special<(bits+15)/16>::type;

#define MAX_BITS_10IEEEFLOAT 0
```